

CONTEXT — How This Submission Was Built, End to End

HACKSIMUL8 2026 · PES University · 2 September 2026 A complete record of the working session: what was decided, what broke, how it was diagnosed and fixed, and why the final design looks the way it does.

Why this document exists. The other documents tell you *what* the solution is. This one tells you *how it got there* — including four genuine defects that were found and fixed along the way. If a judge asks "how did you arrive at this?", the answer is here.

Contents

1. [Timeline](#)
 2. [Starting position — the night before](#)
 3. [The problem statement arrives](#)
 4. [Task 1 — building and verifying the model](#)
 5. [Task 2 — the tuning methodology](#)
 6. [Reading the reference paper — and what it changed](#)
 7. [Task 3 — four defects and their fixes](#)
 8. [Task 4 — the failure that produced the key insight](#)
 9. [Key decisions and the reasoning behind them](#)
 10. [What is solid and what is provisional](#)
 11. [Lessons that generalise](#)
-

1. Timeline

Time	Event
1 Sept, evening	Course material prepared; discovered MATLAB was not installed and the MathWorks account had no licence linked at all
2 Sept 08:29	Problem statement PDF received and parsed
08:33 – 08:40	Complete solution package written (11 MATLAB files) — before MATLAB existed on the machine
08:45	Licence resolved; R2026a installer downloaded (271 MB)
09:02	Installation complete; smoke test — 9 passed, 0 failed
09:06	Task 1 and Task 2 first successful run
09:14	Simulink model built programmatically
09:17	Feedback linearisation verified: 2 μm sag at 17° tilt
09:37	Task 3 data generation launched
10:09	Task 3 FAILED — 0 of 150 usable samples
10:41 – 10:55	Dataset re-labelled under a corrected cost function
11:00 – 11:05	Reference paper obtained and read
11:11	Task 2 re-run under the final cost; gains changed 32.84 → 30.18
11:12 – 11:25	Task 3 regenerated against the corrected baseline
11:32	Task 4 FAILED — mean –168% (worse than no adaptation)
11:36	Simulink model rebuilt with final gains
11:47 – 11:56	Task 4 re-run with the diagnosis applied
11:56	Task 4 succeeded: mean +61.2%

Roughly 3.5 hours of working time, of which about 70 minutes was compute.

2. Starting position — the night before

The event invitation listed three recommended courses (Simulink Onramp, Control System Design, Stateflow Onramp) and named the toolboxes required.

Two blockers were found immediately:

1. **MATLAB was not installed** — no `C:\Program Files\MATLAB`, nothing on `PATH`, no installer present.
2. **The MathWorks account had no licence linked.** License Center reported "*Your MathWorks Account is not currently linked to any licenses.*"

The second blocker explained several downstream symptoms: the Onramp "Start course" button did nothing (the lessons run in a hosted MATLAB Online session), and every course in the Control System Design path showed as locked.

Course notes were written from the live R2026a course pages plus MathWorks documentation (now in [archive/](#)). These turned out to matter less than expected — the problem statement was specific enough that the reference paper became the more useful source.

3. The problem statement arrives

08:29. A three-page PDF. Text extracted with `pypdf`, but **Table 1** and **Figure 1** were embedded images, so they were extracted separately and read directly.

What Table 1 gave us

Parameter	Symbol	Value
Quadcopter mass	m	0.516 kg
Arm length	l	0.225 m
Gravity	g	9.81 m/s^2
Rotor inertia moment	I_M	$3.368 \times 10^{-5} \text{ kg}\cdot\text{m}^2$
Thrust factor	k	$2.996 \times 10^{-6} \text{ N}\cdot\text{s}^2$
Drag coefficient	b	$1.260 \times 10^{-7} \text{ N}\cdot\text{m}\cdot\text{s}^2$
Inertia	I_{xx}, I_{yy}	$4.984 \times 10^{-3} \text{ kg}\cdot\text{m}^2$
Inertia	I_{zz}	$8.958 \times 10^{-3} \text{ kg}\cdot\text{m}^2$

What Figure 1 gave us

A **cross ("plus") configuration**: rotor 1 on $+x_B$, rotor 2 on $-y_B$, rotor 3 on $-x_B$, rotor 4 on $+y_B$, with 1,3 spinning opposite to 2,4. This determined the entire rotor-to-input mapping.

The reading of the four tasks

Task	Literal requirement	What is actually being assessed
1	model in state space	correctness, and whether you can <i>prove</i> it
2	PID + <i>"neat description of the methodology... use the best method available"</i>	the tuning method, not the gains
3	generate data + train an ML model	the dataset design — what are the features
4	test cases with comparisons and merits	honest, quantified comparison

Task 2's wording was read as the highest-value opportunity. It explicitly grades *methodology*. Most teams were expected to tune by trial and present a plot.

Writing code before MATLAB existed

Between 08:33 and 08:40 the complete solution package was written — 11 MATLAB files covering all four tasks — while the installer was still being sorted out. This was a deliberate bet: the physics does not depend on having MATLAB open, and the alternative was idle waiting. It paid off; the first run at 09:06 was 20 minutes after installation completed.

4. Task 1 — building and verifying the model

The derivation

Rotor forces. Each rotor produces thrust $f_i = k \cdot \omega_i^2$ along $+z_B$ and reaction torque $\tau_i = b \cdot \omega_i^2$ about z_B . Applying $\tau = r \times F$ with the Figure 1 geometry gives the four virtual inputs:

$U1 = k(\omega_1^2 + \omega_2^2 + \omega_3^2 + \omega_4^2)$	total thrust
$U2 = k(\omega_4^2 - \omega_2^2)$	roll $\rightarrow \tau_\phi = 1 \cdot U2$
$U3 = k(\omega_3^2 - \omega_1^2)$	pitch $\rightarrow \tau_\theta = 1 \cdot U3$
$U4 = b(\omega_1^2 + \omega_3^2 - \omega_2^2 - \omega_4^2)$	yaw torque

Translational dynamics. Body-frame thrust $[0, 0, U1]^T$ rotated into the inertial frame by the ZYX Euler matrix, plus gravity.

Rotational dynamics. Euler's equations with a diagonal inertia tensor, plus the gyroscopic term from the spinning rotors.

State vector: 12 states, two per degree of freedom.

Linearised about hover: the system decouples into four independent double integrators.

The verification approach — and why it mattered

Rather than assert the algebra was right, the nonlinear model was **finite-differenced** and compared entry-by-entry against the hand-derived A and B matrices:

```
max|A - A_numerical| = 1.636e-12
max|B - B_numerical| = 3.122e-10
rank(ctrb) = 12 of 12
hover residual ||x|| = 0.000e+00
```

This later proved its worth: when the Task 2 gains changed at 11:11 and the model had to be rebuilt, `verify_all` confirmed in two minutes that nothing had broken.

The key structural finding: all twelve open-loop poles sit at the origin. The quadcopter is marginally stable at best and cannot fly without feedback. This became the motivation for everything downstream.

5. Task 2 — the tuning methodology

The central design decision: feedback linearisation

The altitude dynamics are $\ddot{z} = -g + (U1/m) \cdot \cos \phi \cdot \cos \theta$. Tilt costs lift — at 17° of pitch, 4.47% of it.

The obvious approach is to linearise about hover and treat tilt as a disturbance. **We instead cancel it exactly.** Commanding

$$U1 = m \cdot (g + u_z) / (\cos \phi \cdot \cos \theta)$$

makes $\ddot{z} = u_z$ **exactly**, at any tilt angle — algebraically, not approximately.

Verified across a sweep: 0.000034 m sag at 8.6°, 0.000134 m at 17.2°, 0.000298 m at 25.8°. In the Simulink model, **0.000003 m**.

Four tuning methods, presented as a progression

Method	What it optimises	Result
A — analytical pole placement	closed-loop poles, closed form	ITAE 0.5311
B — <code>pidtune</code>	a linear approximation	ITAE 11.1617
C — <code>pidtune</code> at a target crossover	same, faster	ITAE 5.0895
D — ITAE optimisation on the nonlinear model	what actually flies	ITAE 0.0840

Method D is the answer to "use the best method available": A, B and C all optimise an approximation, while D optimises the real system including saturation, anti-windup and the 100 Hz discrete controller.

A trap found in Method A

The first version designed K_p and K_d as a PD ($\zeta = 0.9$) and then added K_i afterwards. That produced **20% overshoot with no settling** — integral action moves the closed-loop poles, destroying the damping just designed for.

Fix: place all three poles simultaneously via $K_d = 2\zeta\omega_n + p$, $K_p = \omega_n^2 + 2\zeta\omega_n p$, $K_i = \omega_n^2 p$.

Method A still overshoots 26%, and the explanation is instructive: **pole placement controls poles, but a PID also introduces a zero** at $s = -K_i/K_p$. A slow zero causes overshoot regardless of pole locations.

Method D avoids this because it optimises the measured response, in which zeros are automatically accounted for.

An observation that generated a testable prediction

Method D drove **K_i to exactly zero**. The reason: feedback linearisation cancels gravity via feed-forward, so in the nominal case there is no steady-state error for integral action to remove — it would only add overshoot.

The consequence was noted at the time: with $K_i = 0$ the controller is *not robust* to an unknown payload. This produced a prediction, written down before any Task 3 data existed:

The ML model should learn to reintroduce integral action when it detects a payload.

Section 7 records how that prediction was confirmed.

6. Reading the reference paper — and what it changed

11:00. The paper — Mien & Tu (2024), IJRCS 4(4) — sits behind a Cloudflare bot check. The user cleared it manually and saved the PDF; it was then parsed locally.

Four findings, each of which changed something

- (a) Their Table 1 is identical to ours.** All eight parameters match. Confirms our inputs.
- (b) Their Eq. (6) includes air-resistance terms A_x, A_y, A_z** that Table 1 never quantifies — in their paper or in our problem statement.
- This was the important one.** Our Task 2 documentation had been claiming, flatly, that "*Ziegler–Nichols is undefined for this plant*". The paper **applies ZN successfully** and publishes measured values ($k_{th} = 124.99$, $\tau_{th} = 3.52$ s).

The resolution: with $A_z \neq 0$ the altitude plant is $1/(s(ms+A_z))$ — one pole off the origin, so a finite ultimate gain exists. With $A_z = 0$, which is the only consistent reading of Table 1, it collapses to a pure double integrator and ZN becomes degenerate.

The claim was rewritten in both the code and the documentation to state the qualification explicitly. An unqualified version would have been caught by any judge who had read the reference.

- (c) Their tuning formulas were extracted and verified.** Applying their Eq. (24) and Eq. (25) to their published k_{th} and τ_{th} reproduces their published Table 2/3 altitude gains to **under 4×10^{-5}** :

Method	Their published	Our computed
Ziegler–Nichols	74.9940, 42.6102, 32.9974	identical
Tyreus–Luyben	56.8136, 7.3365, 31.7435	identical

That verification was done *before* any comparison, to confirm the paper had been read correctly.

- (d) Their altitude control law, Eq. (27), is a plain PID** — no gravity feed-forward, no tilt compensation. This made our feedback linearisation a citable differentiator rather than just a design preference.

The benchmark this produced

All four designs simulated on the identical plant, each in the configuration its author intended, stepping to the paper's own setpoint of 2.0 m:

Design	Ts [s]	OS [%]	ITAE	Tilt sag [m]
Ziegler–Nichols (Mien & Tu)	4.610	12.13	1.7259	0.004522
Tyreus–Luyben (Mien & Tu)	10.300	4.96	6.6841	0.034002
MATLAB PID Tuner (Mien & Tu)	1.490	0.00	0.3319	0.008033
Ours	0.980	0.00	0.1874	0.000134

Cross-check: their paper reports Tyreus–Luyben achieving "steady-state error of less than 1%". Our simulation of their gains gives **0.592%** — inside their stated bound, which independently confirms the reproduction.

7. Task 3 — four defects and their fixes

This was the hardest part of the session. Four separate defects, each found by a result that did not make sense.

Defect 1 — the controller could see the answer

Symptom (10:09): after a 32-minute run, `Usable samples: 0 of 150`.

Diagnosis: `sim_altitude` used a single mass `P.m` for **both** the plant and the controller's feedback linearisation. So "adding a payload" also told the controller about it, and the feed-forward compensated perfectly. Payload had no effect on the response, feature 6 (thrust ratio) read exactly 1.000 every time, and there was nothing to learn. The optimiser then wandered to nonsense gains, and the sanity filter correctly rejected all 150.

Fix: split into `m_ctrl` — the mass the *controller assumes* — and the plant's true mass.

Verification: immediately after, a $\times 1.8$ payload produced 21% steady-state error and feature 6 read **1.736 $\approx$ the true 1.8**. The signal existed.

This is the single most important fix in the project. Without it there is no Task 3.

Defect 2 — the optimiser brute-forced instead of integrating

Symptom: with mass unknown, `fminsearch` returned **$K_p \approx 1978$** .

Diagnosis: it discovered it could mask a steady-state error with enormous proportional gain rather than using integral action. Such gains are physically useless — they amplify sensor noise and sit permanently in saturation.

Fix: added a gain-regularisation term $2e-4 \cdot (K_p^2 + K_d^2)$ to the cost, expressing the real engineering constraint rather than an arbitrary cap.

Defect 3 — `fminsearch` never explored K_i

Symptom: even after Defect 2 was fixed, K_i came back at ≈ 0 for every condition, and steady-state error persisted.

Diagnosis: `fminsearch` builds its initial simplex by perturbing each component of the starting point. A component starting at **exactly zero** gets an almost invisible initial step, so integral action was never meaningfully explored.

Fix: seed K_i at 5.0 rather than 0.

Result — this is the project's headline finding:

Payload	Optimal Ki	Final altitude
×1.0	0.00	1.0171
×1.4	5.50	1.0000
×1.8	11.01	0.9999
×2.2	16.05	1.0000

Ki is exactly zero at nominal and rises almost linearly with payload excess — precisely the prediction made in §5 before any data existed. Steady-state error goes from 36 cm to zero; ITAE improves up to 31×.

A numerical artefact had been hiding the best result in the project.

Defect 4 — overshoot was under-weighted

Symptom: the first Task 4 run showed the learned controller flying to **1.62 m on a 1.00 m command**.

Initial diagnosis was wrong. The cost function's overshoot weight (0.05) was blamed, and the dataset was re-labelled at 0.30. Measuring the effect showed the labels were **never the problem** — optimal controllers in the training set averaged 1.0% overshoot, max 7.1%.

The relabelling helped anyway, for a different reason: warm-starting plus a second seed per condition produced better-converged, more self-consistent optima. Cleaner labels are more learnable, and mean held-out R^2 rose from 0.395 to 0.589.

The real cause of the 1.62 m overshoot was the *model's prediction*, not the training targets — which §8 addresses.

A consistency issue found late

At 11:11, re-running Task 2 under the corrected cost changed the gains from $K_p = 32.8366$ to $K_p = 30.1847$ — an 8% shift.

That mattered because Task 3's training data used the old baseline for its identification flights. Rather than argue 8% was negligible, **Task 3 was regenerated from scratch** against the corrected baseline (13 minutes), and the Simulink model was rebuilt so that code, model and documentation all carry identical gains.

8. Task 4 — the failure that produced the key insight

The failure

11:32. With the model setting all three gains:

```
TC1 predicted Kp = 0.050   ITAE 152.046 vs fixed 18.762   -710.4%   overshoot 353%
TC4 predicted Kp = 0.050   ITAE  31.074 vs fixed 14.855   -109.2%
TC5 predicted Kp = 0.050   ITAE  29.577 vs fixed 12.173   -143.0%
MEAN: -168.3%
```

The ML controller was **substantially worse than doing nothing**.

The diagnosis

0.050 is the safety clamp's floor. The model was predicting **negative Kp** on test conditions outside the training grid — catastrophic extrapolation — and the clamp caught the sign but not the magnitude. With essentially no proportional action, the controller barely responded.

The fix was indicated by the Task 3 regression accuracy, which had been reported honestly all along:

Gain	R ²	Interpretation
Ki	0.70 – 0.87	uniquely determined — a given steady-state error needs a specific integral strength
Kp	0.21 – 0.52	many (Kp,Kd) pairs give near-identical cost
Kd	0.14 – 0.37	same — the cost surface has a flat valley

Kp and Kd labels are intrinsically noisy. No model can predict noise.

And the oracle confirmed adapting them was unnecessary: for TC1 it wanted Kp = 20.2 against our baseline 30.2 (close), while Ki went 0 → 12.2. **Ki is where the entire benefit lies.**

The fix

Adapt only Ki. Hold Kp and Kd at their Task 2 values. Bound the prediction to the range observed in the training labels, so the model interpolates but never extrapolates.

BEFORE: mean -168.3%
AFTER: mean +61.2%

Test case	Improvement	Predicted Ki	Oracle Ki
TC1 heavy payload ×1.8	+80.7%	11.43	12.23
TC2 strong wind + gust	−0.0%	0.00	19.52
TC3 large step + tilt	+73.4%	2.36	3.11
TC4 combined worst case	+70.2%	8.03	8.39
TC5 noise + payload	+81.7%	6.84	8.39

On four of five, predicted Ki lands within ~20% of the oracle's — from three seconds of observation, with no knowledge of the payload.

The fix was methodological, not numerical. It was not hyperparameter tuning. It was asking *which parameters does my data actually identify?*, reading the answer off the R² values, and restricting the model accordingly.

The one unresolved case

TC2 shows exactly 0.0% because the model predicted Ki = 0 — it declined to adapt.

This is explainable rather than mysterious. TC2 is the *lightest* payload case (×1.1), so the thrust-ratio feature read near-nominal and the model correctly inferred no payload correction was needed. What actually hurts

TC2 is **wind**, and the feature set identifies mass well but wind poorly.

The named next step: add a wind-discriminating feature — most plausibly the low-frequency component of the thrust residual after removing the mass estimate.

This is left as a documented limitation rather than patched, because the model declining to act when it cannot identify a condition is *better* behaviour than guessing confidently and getting it wrong — which is exactly what the previous version did.

9. Key decisions and the reasoning behind them

Decision	Alternative	Why
Feedback-linearise the altitude channel	linearise about hover (what the paper does)	exact at any tilt; measured 253× less sag
Adapt only Ki	adapt all three gains	Kp/Kd labels are intrinsically noisy; adapting them gave −168%
Infer mass from thrust ratio	feed mass in as a model input	in flight you do not know the payload; the latter is gain scheduling, not self-tuning
Compare four model classes on held-out data	assume a neural network	the deepest network scored <i>worse than linear</i> ; selection caught it
Include an oracle in Task 4	compare only against the fixed PID	"we beat the baseline" only proves the baseline was weak
Set $A_x = A_y = A_z = 0$	invent plausible drag values	Table 1 gives no values — zero is the only consistent reading
Regenerate Task 3 after the 8% gain change	argue 8% was negligible	slides must match what the code produces live
Bound predictions to the training range	rely on a fixed safety clamp	the clamp caught the sign but not the magnitude

10. What is solid and what is provisional

Solid — measurements that hold regardless

- Table 1 parameters and the derived quantities
- $\max|A - A_{\text{numerical}}| = 1.636\text{e-}12$; rank(ctrb) = 12; all poles at origin
- Final gains $K_p = 30.1847$, $K_i = 0$, $K_d = 10.3994$; $T_s = 0.98$ s; OS = 0.00%
- Tilt sag sweep: 0.000034 / 0.000134 / 0.000298 m at 8.6° / 17.2° / 25.8°
- The benchmark against Mien & Tu, including the 0.592% cross-check
- Ki versus payload: 0 → 5.50 → 11.01 → 16.05
- Correlations: thrust ratio vs Ki $r = +0.840$; mass ratio vs Ki $r = +0.785$
- Task 4 test-case results: mean +61.2%

Provisional — would change if the problem framing changed

- The selected model class and its exact R^2 values (depend on the current label set)
- The cost-function weights in `alt_cost.m`
- The decision to adapt only K_i (correct for *this* feature set; a wind feature could change it)
- The sampled operating envelope

If `alt_cost.m` or the Task 2 gains change, Tasks 3 and 4 must both be re-run. This happened once during the session and is the reason the dependency chain is documented in `RUNBOOK.md`.

11. Lessons that generalise

- 1. A result that is too clean is a bug report.** `0 of 150 usable samples` was obviously broken. But before that, the controller compensating *perfectly* for every payload looked like success — it was actually the controller being handed the answer.
- 2. Report what your data can and cannot identify.** The R^2 split (K_i 0.70+, K_p/K_d ~0.2–0.5) was visible from the first training run. Acting on it — rather than treating it as a number to improve — was what turned –168% into +61%.
- 3. Verify against the source before comparing to it.** Reproducing the paper's published gains to 4×10^{-5} came *before* any performance comparison. Without that step, a discrepancy could mean either "we are better" or "we misread their method".
- 4. Consistency beats marginal accuracy.** The 8% gain change was arguably negligible. But slides that disagree with a live demo are worse than slightly suboptimal gains.
- 5. Numerical artefacts hide real physics.** `fminsearch` not exploring a component that starts at zero is a mundane implementation detail. It was concealing the best result in the project.

Appendix — what to read next

Document	Contents
<code>RUNBOOK.md</code>	How to run everything; every figure explained — which script made it and how
<code>TASK1_final.md</code>	Model derivation, state space, verification, presentation script, Q&A
<code>TASK2_final.md</code>	Feedback linearisation, four tuning methods, paper benchmark, Q&A
<code>TASK3_final.md</code>	Dataset design, model selection, defects, Q&A
<code>TASK4_final.md</code>	Self-tuner, five test cases, why only K_i is adapted, Q&A
<code>README_HANDOFF.md</code>	Index and file guide

Repository: <https://github.com/VismayHS/H8>